Less is Actually More!

Less (which stands for Leaner Style Sheets) is a backwards-compatible language extension for CSS. Originally written in Ruby, it was later ported to JavaScript.

Created in 2009 by Alexis Sellier, Less or Leaner Style Sheets was inspired by Sass. It has a leaner feature set than Sass, and a syntax closely matching CSS, which Sass did not have at that time. In May 2012, Alexis turned over control and development of Less to a core team of contributors who now manage, fix and extend the language. The main difference between Less and other CSS pre-compilers is that the former allows real-time compilation via *less.js* by the browser.

Variables

As the name states, Less helps us to control commonly used values at a single place. We can see many values repeated in our CSS files very often. For example:

```
a,
.link {
  color: #428bca;
}
.widget {
  color: #fff;
  background: #428bca;
}
```

Variables make code easier to maintain by giving a way to control those values from a single location:

```
// Variables
@link-color:        #428bca;
@link-color-hover:  darken(@link-color,
10%);

// Usage
a,
.link {
  color: @link-color;
}
a:hover {
  color: @link-color-hover;
}
.widget {
  color: #fff;
  background: @link-color;
}
```

Mixins

Mixins are a way of including ('mixing in') a bunch of properties from one rule-set into another one. Say, we have the following class:

```
.bordered {
  border-top: dotted 1px black;
  border-bottom: solid 2px black;
}
```

Now suppose we want to use these properties inside other rule-sets. Well, we just have to drop in the name of the class where we want the properties, in this way:

```
#menu a {
  color: #111;
  .bordered();
}

.post a {
  color: red;
  .bordered();
}
```

The properties of the *.bordered* class will now appear in both *#menu a* and *.post a*. In normal CSS, we would have had to repeat this multiple times. As you can see, variables as well as mixins help us avoid having repeated values in the file.

Nesting

Less gives you the ability to use nesting instead of, or in combination with, cascading. For example:

```
#header {
  color: black;
}
#header .navigation {
  font-size: 12px;
}
#header .logo {
  width: 300px;
}
```

In Less, we can also write it this way:

```
#header {
  color: black;
  .navigation {
    font-size: 12px;
  }
  .logo {
    width: 300px;
  }
}
```

The resulting code is more concise and mimics the structure of your HTML at-rules. For example, *@media* or *@supports* can be nested in the same way as selectors. The at-rule is placed on top, and the relative order against other elements inside the same rule set remains unchanged. This is called bubbling.

```
.component {
  width: 300px;
  @media (min-width: 768px) {
    width: 600px;
    @media  (min-resolution: 192dpi) {
      background-image: url(/img/
retina2x.png);
```